A3 Part 4: Individual Reflection

Prototype: Count Down Under · **Cycle:** July 28 to August 17, 2026 · **Group of four**

How my practice held across the longer cycle

The part that held was the cold start. Day one was the repo skeleton: initial check-in, default packages, CoreUtils, the `_Project` structure, fonts, and the scenes brought across from the jam version. That took an afternoon, and by the end of July 28 the group had somewhere to put work. I have done that opening enough times that it is close to automatic, which is itself the argument for making it a template instead of retyping it every project.

The part that did not hold was pacing. I committed on eleven of the twenty one days in the cycle, and four of those days carry 56 of my 89 commits. Roughly two thirds of my work landed on four days.

That is the shape of how I work. I am slow to start and productive once I get going, so what drove my schedule was whenever momentum happened to arrive. The sprint boundaries did not. On a four day jam this is invisible, because the whole project fits inside one or two of those productive stretches. Across three weeks it becomes the defining feature of the cycle.

The other half of pacing is rest, and I was bad at it in both directions. Some days I had to talk myself out of working, because the laundry had been ignored for a week and the room needed sweeping and none of that stops being true because a build is due. Giving myself permission to not work is a skill I do not have yet, and on a three week cycle it matters more than it does on a jam, because there is no finish line close enough to sprint to.

What worked

Branch per feature, with one integration point. The group ran fifteen named feature branches and merged them into a shared staging branch. For four people in the same Unity project that structure did its job and nobody lost work to a merge.

One afternoon of observation redirected the entire final week. On August 10 a former instructor reviewed the build and, later that day, a teammate took notes while his brother played it. Everything shipped afterwards traces back to that day: the crocodile and lunge attack notes become the croc and roo animation pass on the 13th, "analogue controls delay in directional change" becomes a Cinemachine camera with lookahead on the 13th and a camera deadzone after that, the notes on chromatic aberration, colour correction and 2D spot lights become the lighting and post-processing work on the 15th and 16th, and "reveal that the tunnels are where you can go" becomes a level design update on the 16th and the dawn fade on the 17th. One session, and the last week of a three week project ran on it.

What broke in coordination

I was told to ask before merging, more than once, and kept not doing it. When I had momentum I would pull in whatever people had pushed, integrate it, test a build locally and move on, without a message to anyone. I am inconsistent about answering Discord myself, sometimes out of guilt and sometimes because I do not want to talk to anyone, and I projected that onto everybody else, so a message felt like an imposition rather than the minimum.

The instruction was clear each time I received it. What failed was that it never survived contact with being on a roll. People were uncomfortable with how freely I moved their work around and uncomfortable enough saying so that I found out late. This is the largest coordination failure of the cycle.

Because it is a pattern and not an oversight, I do not want to own the repository on the next project. Removing the ability to merge at will is a guardrail I can rely on when my judgement in the moment has already proven

unreliable, and it costs the team nothing.

I built process nobody asked for. I wrote a naming conventions document at full length, and I set up a GitHub project board with assignments and to-do, in-progress and done columns, so that tracking would be centralised instead of scattered across Discord messages and one in the morning pushes. The reasoning was sound and the volume was not. Both landed as overhead on a group that had not asked for either, and a system nobody opens does the same amount of work as no system. The instinct to organise is worth keeping. The impulse to build the whole thing before discussing it is not.

I was the only person who made a build. Every build in this cycle and every project before it was mine. That is not reluctance to share, I like troubleshooting and I am happy to be the one fixing the compile error at eleven at night, but it means three people finished a three week cycle without ever making a Windows build, switching a target to WebGL, or pushing to itch. Next term is a single project across a whole semester with a larger group, and I want all of us to have made one, including the version that fails, because "I used the same settings and it broke" is where the learning is.

We observed once inside the cycle, on day ten of twenty one. Each sprint was supposed to put its build in front of players. We managed the middle sprint and never again while there was still a sprint left to spend the findings in. Neither observer that day was outside our circle either: one was a former instructor, the other a teammate's brother.

Part of that is that getting people to play is hard, and people are reluctant to hand back negative feedback. Part of it is mine. I find it uncomfortable to watch someone critique work I made, because I hear it as a judgement on the effort rather than on the result. That is my problem to solve and not a reason to test less. I had roommates I never asked.

The outside players arrived on submission day. On 18 August I sat down two people with no connection to the class, one from Foundations and one from administration, and watched them play. They were the first players in the entire cycle who were not a teammate, a teammate's relative, or an instructor, and they produced more usable information in one sitting than I expected: the first instinct to attack with the mouse, X and Z being mixed up constantly, no controls on the main menu, traversal confusion, not wanting to enter bat form every time, and the humour living in the game's description rather than in the game. One died and immediately started again, which is the best signal in any of the notes.

None of it could be acted on. The session that most resembled a real test was the one I ran on the day the work stopped.

I assumed someone else would fix it. By the back half of the cycle everyone was tired, this being the last of a long run of small projects, and I caught myself deciding that a problem sat closer to somebody else's area so they would probably handle it. Sometimes nobody did. The clearest cost is the sightseeing objective: a counter went into the build on August 6, the player on August 10 still did not understand the objective, an unrelated player on August 18 did not understand it either, and it is still not properly implemented. Three independent observations of the same failure, eight days apart, on a to-do that everyone could see and nobody owned.

That failure is not evenly distributed, either. The person on this team producing the most work by volume was doing art at a rate the rest of us were not matching, and I under-communicated with her repeatedly. Effort was not what I owed her. Replies were.

The records never left paper. All three sets of playtest notes are handwritten and none of them are in the repository. We talked through the 10 August results in person, all four of us, and wrote none of it down. No commit message references a playtest, so the link between an observation and the change it caused exists only in memory. Reconstructing that mapping afterwards from commit dates and wording took an hour and parts of it are still guesses.

What I would carry into my next longer build

Talk first, then act. Reading back through everything above, almost every failure in this cycle is the same failure: I did the reasonable thing quickly instead of proposing it slowly. The merge convention, the naming document, the project board, the packages I install because we will probably need them. The fix in every case is one conversation I did not have.

Plan the shape of the work up front, and expect it to move. Something like a Gantt chart, enough to know roughly what has to happen in what order, then reassessed as it goes. Agile rather than waterfall, because waterfall assumes you never have to go back, and in practice someone tells you a piece is done, you believe them and move on, and then they become unavailable and you are swimming back upstream.

Make the cold start an actual template, over the break. Bare bones only, and anything beyond bare bones gets discussed before it gets installed.

One repo per project, not one per term. Everything this term lived in a single semester repo across four course branches, which makes it hard for an instructor to mark and drags irrelevant history along with every assignment.

Hand the build to somebody else in sprint one. I would still like to be the one who gets things rolling, because I am fast at it, but I do not need to be the only one who can.

Right-size documentation: stubs first, expand on request.

Get outside playtesters during the cycle, not on the day it ends. I already know this works, because I did it eight days too late. Another classmate is useful because they have hit the same problems and can suggest fixes. Someone from outside the program is useful for a different reason: they do not know what they are supposed to be impressed by. They do not even have to play it. Watching someone play and asking "why did you do it that way" works the same way it does watching someone fix a car or play a sport, where the question from the person who does not know the routine is the one that exposes the assumption.

Put playtest notes in the repo, dated, beside the sprint they belong to, and mark builds with git tags the way I have on previous projects, where 0.3, 0.4 and 0.5 point at the commits that produced each published build. It costs nothing and it makes the history readable later.